

Line of sight, area di visibilità e fog of war nei tattici XCOM-like

Executive summary

Nei tattici a turni in stile *XCOM*, la soluzione più robusta non è un singolo algoritmo ma una **pipeline a strati**:

geometria di gameplay → LoS/FOV deterministica → stato “visibile/esplorato/non visto” → texture o buffer di visibilità → shader di presentazione → effetti secondari come dissolve, silhouette e nebbia volumetrica.

Questa separazione è fondamentale. La linea di vista che decide se un nemico esiste dal punto di vista del gameplay non dovrebbe dipendere da blur, TAA, particle fog, occlusion query della GPU o altri effetti visivi. In *XCOM 2*, per esempio, la LoS ha conseguenze esplicite sul gameplay: il manuale mostra quanti nemici sono nella linea di vista, evidenzia con l'icona dell'occhio le celle nella LoS avversaria e usa quella stessa informazione per determinare la perdita del concealment. ¹

La mia raccomandazione generale per un nuovo XCOM-like è:

Shadowcasting o visibility-grid per il FOV di area + raycast 3D per verifiche critiche di LoS/Line-of-Fire + due stati persistenti `VisibleNow` / `Explored` + texture R8/RGB8 world-space per la GPU + edge smoothing/SDF + filtro temporale solo grafico.

È un compromesso molto forte perché il calcolo tattico resta discreto, riproducibile e debuggabile, mentre il rendering può essere molto più morbido e cinematografico. libtcod, per esempio, espone nello stesso framework sia raycasting sia recursive/symmetric shadowcasting e vari algoritmi permissivi/restrittivi, mostrando quanto sia normale trattare il FOV come un problema separato dal renderer. ²

Per mappe geometricamente arbitrarie ma sostanzialmente planari, i **visibility polygon** sono più precisi di una griglia: in geometria computazionale la regione di visibilità da un punto è proprio l'insieme dei punti raggiungibili da un segmento non ostruito; esistono algoritmi specifici efficienti per calcolare tale poligono. ³

Le **hardware occlusion query** e la **precomputed visibility** dei renderer sono invece da considerare acceleratori grafici, non sostituti della LoS tattica. Unreal, per esempio, descrive le hardware occlusion query come sistema di dynamic occlusion culling della scena e la precomputed visibility come strumento per evitare di renderizzare Actor non visibili dalla camera. ⁴

Dal lato estetico, un effetto convincente nasce solitamente da quattro componenti:

Strato	Funzione	Aggiornamento ideale
VisibleNow	ciò che la squadra vede davvero	evento tattico
Explored	ciò che è stato visto almeno una volta	persistente
DisplayVisibility	versione filtrata temporalmente	ogni frame
VFX di frontiera	dissolve, particelle, volumetrico, glow	ogni frame

Un'implementazione UE4 pubblica esplicitamente costruita per ottenere una fog grid-based simile a *XCOM 2* usa esattamente un approccio del genere: una Render Target dove ogni cella visibile diventa un pixel, un materiale di proiezione e una seconda Render Target per interpolare la transizione nel tempo. L'autore nota anche un artefatto molto importante: il semplice filtering/blur è economico ma può far "trapassare" la nebbia attraverso muri sottili. ⁵

Modello architetturale: separare gameplay e grafica

Prima di parlare di shader conviene fissare tre concetti diversi.

Line of Sight, LoS significa generalmente una verifica tra due posizioni: "A vede B?". È il caso tipico di un raycast da un punto di osservazione verso il bersaglio.

Field of View, FOV è invece l'insieme delle posizioni visibili da A. Shadowcasting, visibility polygons e rasterizzazione di coni appartengono soprattutto a questa categoria.

Fog of War, FoW è lo stato informativo presentato al giocatore. Può distinguere almeno tra spazio attualmente visibile, spazio già visitato ma non più osservato e spazio totalmente sconosciuto.

La distinzione fra LoS puntuale e area visibile emerge anche nelle librerie FOV: libtcod dispone di algoritmi distinti che vanno dal raycast verso il perimetro della regione al recursive shadowcasting, al symmetric shadowcasting e alle varianti permissive. ⁶

Una rappresentazione estremamente pratica è:

$$V_t(x) \in [0, 1]$$

per la visibilità corrente della cella o posizione x , e:

$$E_t(x) = \max(E_{t-1}(x), V_t(x))$$

per la memoria delle zone esplorate.

Lo **stato logico** può rimanere binario. Il valore continuo è utile soprattutto dopo, nel renderer.

```
Gameplay:
0 = invisibile
1 = visibile

Memoria:
0 = mai esplorato
```

1 = esplorato almeno una volta

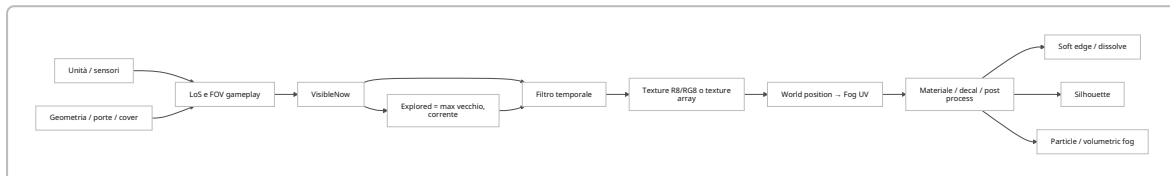
Rendering:

0.00 = completamente occultato

0.25 = esplorato ma non attualmente visibile

1.00 = visibile

Il flusso che consiglierei è questo:



Il punto cruciale è che la freccia non dovrebbe andare al contrario: una particella o un pixel sfumato **non dovrebbe decidere** se un alieno è stato scoperto.

Questo è particolarmente importante con l'occlusion culling. Unreal usa view-frustum culling e hardware occlusion query per decidere quali Actor convenga renderizzare dalla camera; si tratta di una decisione di rendering legata al punto di vista, non della semantica tattica “questa unità può vedere quella unità”. ⁷

La stessa cautela vale per la precomputed visibility. Unreal genera celle offline e memorizza quali Actor siano visibili dalle varie celle, scambiando tempo di rendering per memoria e tempo di build; Epic la raccomanda soprattutto per ambienti relativamente statici, movimento della camera limitato e aree sostanzialmente 2D. È molto interessante come acceleratore, ma non è automaticamente una LoS gameplay corretta in presenza di porte, distruzione o oggetti dinamici. ⁸

In un tattico multi-floor utilizzerai inoltre un livello separato per piano:

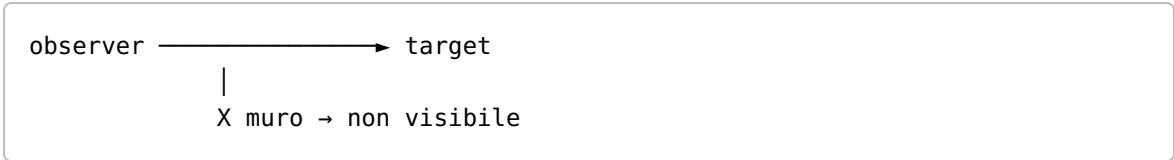
```
FoWTextureArray
├─ slice 0 : basement
├─ slice 1 : ground
├─ slice 2 : first floor
└─ slice 3 : roof
```

invece di affidarmi solo alla proiezione XY. Questo evita che un'unità al piano terra “buchi” accidentalmente la nebbia del primo piano.

Metodi per calcolare line of sight e area visibile

Raycasting

Il raycasting è l'approccio più intuitivo:



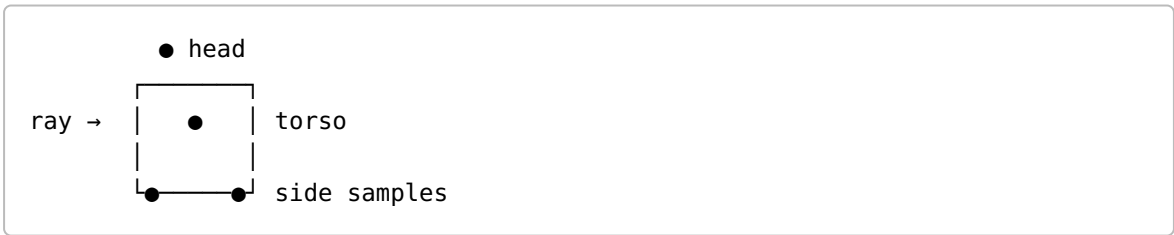
Per ogni bersaglio o punto campionato si interroga la geometria lungo il segmento osservatore→target.

Su una griglia, il raycast può essere discretizzato con Bresenham/DDA; in uno scenario tridimensionale si possono invece usare collision query del motore. libtcod include, per esempio, un algoritmo FOV "basic" che effettua raycast dal giocatore verso le celle del perimetro dell'area. ⁹

Punti di forza: lavora bene con geometria 3D, distruzione e ostacoli arbitrari; è concettualmente semplice; si combina naturalmente con PhysX/Chaos/Unity Physics o un BVH custom.

Limite principale: se vuoi conoscere tutte le celle visibili, il numero di raggi aumenta rapidamente. Inoltre la risposta dipende dal modello geometrico: ray verso centro cella, testa del personaggio, torso, angoli della hitbox? Un singolo ray può produrre risultati “pixel perfect” che sembrano ingiusti tatticamente.

Una soluzione pratica nei tattici è usare **più sample per il bersaglio**:

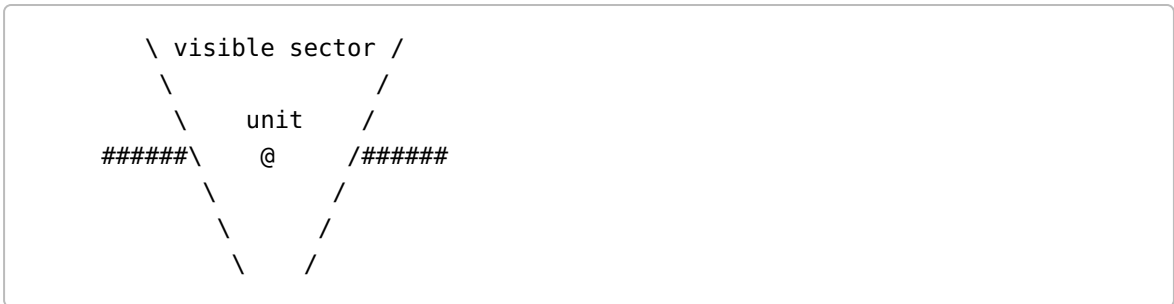


oppure usare un algoritmo FOV discreto per la visibilità generale e lasciare i raycast solo per i casi dove precisione e geometria verticale contano davvero.

Shadowcasting

Il recursive/symmetric shadowcasting non lancia necessariamente un raggio indipendente verso ogni cella. Divide la griglia in quadranti/ottanti e mantiene intervalli angolari o "slope" ancora illuminati mentre procede verso l'esterno.

Il concetto è:



Quando un ostacolo intercetta una porzione dell'intervallo, quella porzione viene sottratta dalle regioni ancora da visitare. Le implementazioni ben costruite fanno un lavoro vicino a una visita delle celle interessate piuttosto che una moltitudine di raycast indipendenti. ¹⁰

Per un gioco tattico sceglierei in particolare una variante **simmetrica**. L'implementazione descritta da Albert Ford è esplicitamente progettata affinché la visibilità abbia proprietà consistenti e simmetriche; è molto utile quando vuoi evitare situazioni del tipo "A vede B, ma dalla stessa geometria B non vede A".

¹¹

È eccellente per:

- mappe a tile;
- FOV di molti soldati;
- aggiornamenti dopo ogni movimento;
- livelli ortogonali;
- anteprima della visibilità da una casella futura.

È meno adatto quando muri e aperture devono essere trattati con precisione sub-tile o quando la verticale è una parte importante della geometria.

Flood fill

Il flood fill visita celle contigue a partire da una sorgente:

```
queue.push(origin)

while queue not empty:
    c = queue.pop()

    for neighbor in c.neighbors:
        if traversable(neighbor) && !visited(neighbor):
            visited(neighbor) = true
            queue.push(neighbor)
```

Da solo **non è un vero algoritmo di LoS**: misura connettività, non la presenza di un segmento ottico non ostruito.

È però super utile come secondo livello, per esempio:

```
LoS supera la porta?
|
├ sì → flood-fill la stanza
└ no → stanza nascosta
```

oppure per reveal "room based", dungeon, edifici con portali, calcolo dell'insieme di celle potenzialmente interessanti prima di eseguire test geometrici più costosi.

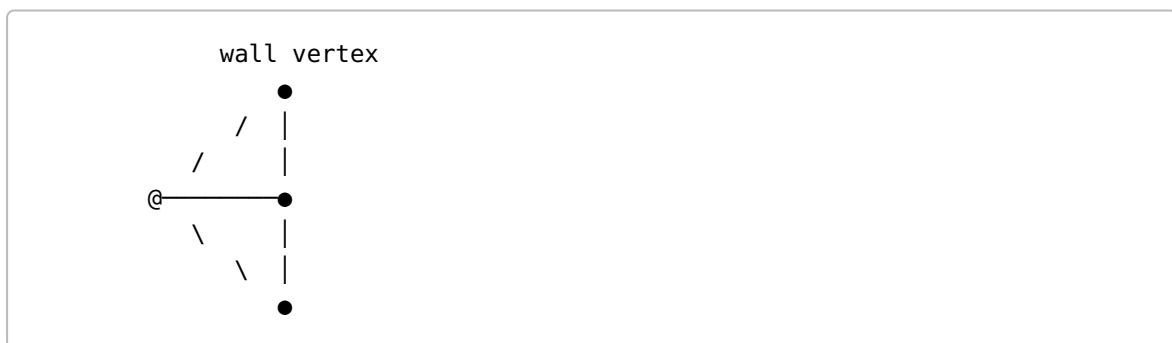
La complessità è lineare rispetto alle celle/conessioni effettivamente visitate, quindi su griglie tattiche è molto economico.

Visibility polygon

Per una geometria top-down composta da segmenti e poligoni 2D, il visibility polygon è forse il modello geometricamente più pulito.

Formalmente, due punti sono visibili quando il segmento che li collega rimane all'interno della regione percorribile/non attraversa gli ostacoli; l'insieme dei punti visibili da un osservatore forma la sua regione o poligono di visibilità. Il paper *Efficient Computation of Visibility Polygons* descrive e confronta diverse implementazioni di algoritmi per questo problema. ³

Una tecnica pratica è un angular sweep:



Si ordinano gli eventi per angolo attorno all'osservatore e si mantiene l'ostacolo più vicino lungo il raggio corrente. Una implementazione open source del metodo descrive proprio una sweep line costituita da un raggio che ruota attorno all'osservatore mantenendo ordinati i segmenti intercettati per distanza. ¹²

La cosa bella lato rendering è che il risultato è direttamente **una mesh**:

```
LoS CPU
  ↓
visibility polygon
  ↓
triangulate fan
  ↓
render into mask
```

Quindi niente ricostruzione complicata: basta rasterizzarlo in una Render Texture.

Il problema arriva con:

- più piani;
- rampe;
- coperture basse;
- finestre;
- unità con altezze diverse;

- oggetti che ostruiscono la visuale solo parzialmente.

In quei casi il visibility polygon 2D diventa un'approssimazione.

Occlusion queries

Le hardware occlusion query chiedono essenzialmente alla GPU se il volume di un Actor ha prodotto pixel visibili dopo il depth test.

Sono fantastiche per evitare draw inutili, ma pessime come fonte primaria della visibilità tattica. In Unreal fanno parte del sistema standard di dynamic occlusion culling insieme al frustum culling. ⁷

I problemi concettuali per un XCOM-like sono:

1. dipendono dalla **camera**, non dagli occhi dell'unità;
2. operano su primitive/bounding volume da rendering;
3. possono comportare latenze GPU↔CPU;
4. non modellano automaticamente regole come range sensore, concealment, smoke tattico o conoscenza persistente;
5. non sono naturalmente adatte a chiedere "cosa vedrebbe il soldato da quella cella dopo che lo sposto?".

Quindi: ottimo ottimizzatore visivo, pessima authority tattica.

Precomputed visibility

L'idea consiste nel dividere l'ambiente in celle e precomputare qualcosa come:

```
Cell A → {Room 1, Room 2}
Cell B → {Room 2}
Cell C → {Room 2, Room 3, Roof}
```

A runtime la risposta può diventare un lookup estremamente veloce.

Unreal implementa una forma ufficiale di questo paradigma con i Precomputed Visibility Volumes: le visibility cell vengono generate offline sopra la geometria statica e memorizzano la visibilità degli Actor da quelle posizioni. Epic indica esplicitamente il trade-off: meno lavoro di occlusion query durante il rendering in cambio di più memoria runtime e build time. ⁸

Per gameplay tattico lo stesso pattern può essere applicato a:

- cella→celle visibili;
- cover point→cover point;
- nav node→enemy observation points;
- room→room attraverso portali.

È fortissimo per livelli statici indoor. Diventa molto meno conveniente quando ogni missione può distruggere muri, aprire passaggi o ricombinare pezzi procedurali.

Navmesh-based visibility

Una navmesh è eccellente come **dominio di campionamento** della visibilità, ma non dovrebbe essere confusa con la visibilità stessa.

Un pattern classico è:

```
Nav polygon
  ↓
sample tactical point
  ↓
compute exposure to enemies
  ↓
visibility / threat cost
  ↓
A* chooses concealed route
```

Il capitolo *Tactical Pathfinding on a NavMesh* di *Game AI Pro* descrive proprio l'idea di assegnare un costo maggiore ai nodi visibili/esposti al nemico e un costo minore a quelli nascosti. Discute inoltre il costo di effettuare molti raycast di visibilità a runtime e l'alternativa di precomputare lookup, evidenziando il trade-off con la memoria. ¹³

Questa tecnica è perfetta per:

- scegliere cover;
- calcolare percorsi stealth;
- mostrare l'esposizione lungo un movimento;
- calcolare "da quale posizione posso vedere quell'area?";
- restringere il numero di raycast alle posizioni tatticamente raggiungibili.

Ma il navmesh codifica soprattutto **dove si può camminare**. Per finestre, parapetti, fumo, differenze di altezza e coperture distruttibili serve comunque uno strato ottico separato.

Rendering della fog: shader, maschere e transizioni

Il passaggio più importante lato grafico è trasformare la visibilità tattica in una risorsa campionabile dalla GPU.

Texture world-space della visibilità

Il pattern più generalizzabile è una texture che rappresenta la mappa.

Esempio: mappa tattica 128×128 celle.

Una texture $R8\ 128 \times 128$ occupa solo 16 KiB senza mipmap; una $RG8$ 32 KiB. Il canale R può contenere `VisibleNow`, G `Explored`.

La conversione world→fog UV è:


```
float2 fogUV = (worldPos.xz - _MapMin.xz) / _MapSize.xz;
```

e successivamente:

```
float2 state = SAMPLE_TEXTURE2D(_FogTex, sampler_FogTex, fogUV).rg;
```

L'implementazione UE4 di minmax usa proprio una Render Target grid-based: converte l'indice della cella in coordinate 2D, disegna un pixel bianco per cella visibile e usa poi la Render Target nel materiale.

14

È l'approccio che sceglierei come default perché è:

- indipendente dalla risoluzione dello schermo;
- indipendente dall'angolo della camera;
- semplice da usare anche sulla minimappa;
- condivisibile da terreno, decal, particelle e materiali;
- facile da salvare per l'esplorazione persistente.

Hard fog

La forma più semplice è:

$$visibility = \begin{cases} 1 & mask > threshold \\ 0 & altrimenti \end{cases}$$

Visivamente produce un confine durissimo.

È ideale quando vuoi:

- stile board-game;
- massima chiarezza;
- costo shader praticamente nullo;
- pixel art;
- debug.

In un tattico 3D realistico però l'effetto tende a sembrare "ritagliato".

Bilinear filtering e blur

Il primo upgrade naturale è filtrare la texture:

hard:



blur:



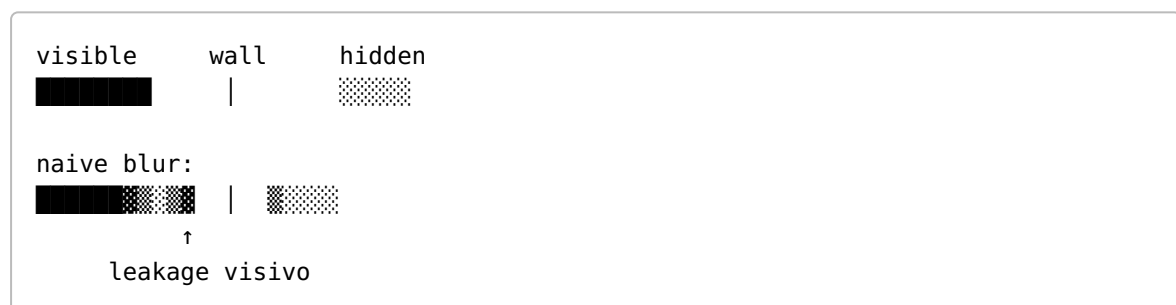
Puoi ottenere il risultato tramite:

- texture bilinear;
- mipmap;
- box blur;
- Gaussian blur;
- separable blur;
- compute shader.

È molto economico a bassa risoluzione. Ma c'è una trappola importante: **blurrare la visibilità non conosce i muri**.

L'articolo UE4 che replica l'estetica XCOM-like nota esplicitamente che il filtering è molto economico ma può far sanguinare la nebbia oltre muri sottili. ¹⁵

Esempio:



Tre rimedi sono migliori:

Barrier-aware blur. Durante il kernel non campionare texel appartenenti all'altro lato di un blocker.

Signed Distance Field. Calcola la distanza dal vero confine di visibilità e usa quella distanza per creare la sfumatura; il bordo diventa stabile anche con zoom diversi.

Supersampling. Invece di un solo texel per tile, rasterizza il FOV a 2×–4× la risoluzione della griglia e poi riduci.

Per un XCOM-like serio preferisco SDF o supersampling a un Gaussian blur aggressivo.

Soft edge con SDF

Supponiamo che `d` sia una signed distance dalla frontiera:

```
float visibility =  
    smoothstep(-_FadeWidth, +_FadeWidth, d);
```

Questo separa due decisioni:

Gameplay boundary



esatta

Graphics boundary



solo visiva

È davvero un bel pattern perché puoi cambiare `_FadeWidth` senza modificare le regole tattiche.

Temporal smoothing

Il secondo ingrediente che fa smettere alla fog di sembrare una maschera tecnica è la transizione temporale.

Minmax mostra una soluzione tramite una **seconda Render Target** nella quale viene progressivamente dipinta/interpolata la maschera corrente. ¹⁵

Una versione frame-rate independent è:

$$\alpha = 1 - e^{-\Delta t / \tau}$$

$$D_t = \text{lerp}(D_{t-1}, V_t, \alpha)$$

dove:

- V_t = stato tattico istantaneo;
- D_t = stato mostrato;
- τ = costante temporale.

Pseudocodice:

```
float response = 1.0 - exp(-deltaTime / tau);  
display = lerp(previousDisplay, targetVisibility, response);
```

Una variante particolarmente efficace nei tattici usa due velocità:

```
reveal: 80–150 ms  
hide:   250–500 ms
```

La comparsa veloce comunica subito “hai scoperto qualcosa”; la scomparsa leggermente più lenta evita pop visivi durante il movimento della camera.

Questi valori sono una scelta artistica, non una regola dell'algoritmo.

Dissolve

Per una frontiera più stylized, usa una noise texture o blue-noise per perturbare il threshold.

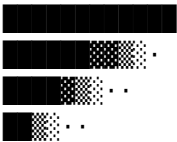
Concettualmente:

$$visible = smoothstep(T - w, T + w, M + N \cdot A)$$

dove:

- **M** = mask;
- **N** = noise;
- **A** = ampiezza;
- **T** = threshold animato.

Visivamente:



È importante applicare il noise **solo nella fascia di transizione**, non su tutta l'area esplorata, altrimenti il terreno “frigge” continuamente.

Puoi poi usare il bordo per emission:

```
float edge =
    smoothstep(t - width, t, noisyMask)
    - smoothstep(t, t + width, noisyMask);

color += edge * _EdgeEmission;
```

In un'ambientazione sci-fi questo produce una transizione molto XCOM-ish senza dover usare particelle vere.

Screen-space e Scene Depth

Invece di applicare la FoW a ogni materiale, puoi fare tutto in un fullscreen/post-process pass:

```
screen UV
↓
scene depth
↓
reconstruct world position
↓
world XY/XZ → fog UV
↓
sample FoW
↓
grade scene
```

Unity Shader Graph espone il nodo **Scene Depth**, che campiona la depth texture della camera alle coordinate screen-space; in URP la depth texture deve essere abilitata e il nodo opera nel fragment stage. ¹⁶

I vantaggi sono enormi:

- nessuna modifica ai materiali del livello;
- una sola pipeline della FoW;
- facile desaturazione delle aree esplorate;
- dissolve e color grading uniformi.

Lo svantaggio è che devi gestire bene:

- sky;
- particelle;
- acqua;
- trasparenti;
- UI 3D;
- oggetti che non scrivono correttamente nella depth.

Depth-based masking

La depth è utile anche per evitare intersezioni brutte fra fog/particelle e geometria.

Unity URP implementa i **Soft Particles** proprio confrontando la particella con il depth buffer: la particella viene sfumata quando si avvicina alla superficie della geometria. ¹⁷

Lo stesso principio può essere riusato per la FoW:

```
float delta = geometryDepth - fogParticleDepth;
float softIntersection =
    saturate(delta * _InvFadeDistance);

fogAlpha *= softIntersection;
```

È particolarmente utile per nebbia bassa o VFX che seguono il confine della zona sconosciuta.

Silhouette highlight

Nei tattici capita spesso di voler vedere un personaggio conosciuto anche se un muro lo copre dalla **camera**, purché il personaggio sia legalmente visibile alla squadra.

Queste sono due visibilità diverse:

```
Gameplay LoS:   squadra → unità
Camera LoS:     camera  → unità
```

La silhouette dovrebbe apparire solo quando:

```
gameplayVisible == true  
AND  
cameraOccluded == true
```

Puoi implementarla con un Custom Depth/ID buffer:

```
SceneDepth    = muro  
ActorDepth    = personaggio dietro muro  
  
ActorDepth > SceneDepth  
↓  
actor è camera-occluded  
↓  
se gameplayVisible → outline
```

È essenziale mettere il gate `gameplayVisible`: altrimenti l'effetto di outline diventa un wallhack e rivela informazioni che la FoW dovrebbe nascondere.

Particle e volumetric fog

Una FoW non deve per forza essere un semplice overlay nero. Una frontiera può essere accentuata con:

- smoke card;
- GPU particles;
- wisps;
- dust;
- volumetric density;
- local fog volumes;
- light scattering.

In Unreal il Volumetric Fog è un vero medium partecipante calcolato nel frustum della camera tramite una volume texture; Epic permette sia controllo globale sia contributi locali tramite particle system. ¹⁸

È però uno strato relativamente costoso. Epic indica che la qualità dipende dalla risoluzione della volume texture e dal numero di slice Z; aumentare `GridSizeZ` migliora la qualità ma aumenta il costo. La documentazione riporta inoltre, come riferimento storico della propria implementazione, circa 1 ms su PS4 a High e 3 ms su GTX 970 a Epic. ¹⁹

In pratica non farei:

```
FoW gameplay = volumetric fog
```

ma:

```
FoW gameplay mask  
+  
piccola fascia volumetrica alla frontiera
```

+
particelle sparse

Così la densità volumetrica può perfino laggere leggermente o essere LODdata senza toccare le regole del gioco.

Pattern shader pronti da adattare

Gli snippet seguenti sono intenzionalmente **HLSL-style pseudocode**: la matematica è direttamente trasferibile a HLSL, GLSL, Unity Shader Graph o Unreal Material Editor, mentre le funzioni concrete di sampling/depth cambiano con la pipeline.

Hard fog con stato explored

```
struct FogState
{
    float current;
    float explored;
};

FogState SampleFog(float3 worldPos)
{
    float2 uv = (worldPos.xz - _MapMin.xz) / _MapSize.xz;

    float2 data =
        SAMPLE_TEXTURE2D(_FogTexture, sampler_FogTexture, uv).rg;

    FogState state;
    state.current = data.r;
    state.explored = data.g;
    return state;
}

float3 ApplyHardFog(float3 sceneColor, float3 worldPos)
{
    FogState f = SampleFog(worldPos);

    if (f.current > 0.5)
        return sceneColor;

    if (f.explored > 0.5)
    {
        // Mantieni leggibile la geometria già scoperta.
        return sceneColor * _ExploredBrightness;
    }

    return _UnexploredColor.rgb;
}
```

La separazione fra current e global/permanent visibility appare anche in VaFogOfWar, un'implementazione open source per UE4 che supporta distintamente current visibility, global visibility e terrain, oltre a ostacoli dinamici/statici e più livelli di altezza. 20

Soft fade con dithering

```
float ApplySoftVisibility(float rawMask, float2 screenUV)
{
    // Bordo spaziale.
    float soft =
        smoothstep(_FogEdgeStart, _FogEdgeEnd, rawMask);

    // Preferibilmente una blue-noise texture tiled.
    float noise =
        SAMPLE_TEXTURE2D(_BlueNoise, sampler_BlueNoise,
                        screenUV * _NoiseScale).r;

    // Dither concentrato intorno alla transizione.
    float edgeBand = 1.0 - abs(soft * 2.0 - 1.0);
    float perturbation =
        (noise - 0.5) * _DitherStrength * edgeBand;

    return saturate(soft + perturbation);
}

float3 ApplyFog(
    float3 sceneColor,
    float visibility)
{
    return lerp(
        _FogColor.rgb,
        sceneColor,
        visibility
    );
}
```

Per alpha-cutout:

```
clip(visibility - _DitherThreshold);
```

Per alpha blend:

```
float fogAlpha = 1.0 - visibility;
```

Nel secondo caso controllerei attentamente l'overdraw; nel primo puoi ottenere una frontiera più economica, soprattutto se il TAA riesce a integrare temporalmente il dithering.

Dissolve animato

```
float ComputeDissolve(
    float visibility,
    float2 worldUV,
    float transition)
{
    float noise =
        SAMPLE_TEXTURE2D(
            _DissolveNoise,
            sampler_DissolveNoise,
            worldUV * _NoiseTiling
        ).r;

    float value =
        visibility
        + (noise - 0.5) * _NoiseAmount;

    return smoothstep(
        transition - _EdgeWidth,
        transition + _EdgeWidth,
        value
    );
}
```

Per un bordo emissivo:

```
float a = smoothstep(
    transition - _EdgeWidth,
    transition,
    value);

float b = smoothstep(
    transition,
    transition + _EdgeWidth,
    value);

float edge = saturate(a - b);

finalColor += edge * _EdgeColor * _EdgeIntensity;
```

Temporal smoothing

```
float SmoothVisibility(
    float previous,
    float target,
    float deltaTime)
{

```

```

float tau =
    target > previous
    ? _RevealTimeConstant
    : _HideTimeConstant;

float alpha =
    1.0 - exp(-deltaTime / max(tau, 0.0001));

return lerp(previous, target, alpha);
}

```

Può essere eseguito in:

- ping-pong Render Texture;
- compute shader;
- fullscreen pass;
- CPU se la griglia è davvero piccola.

L'idea di mantenere una seconda Render Target come stato interpolato è stata dimostrata direttamente nell'implementazione grid-based UE4 di minmax. ¹⁵

Silhouette dietro occluder

Assumendo depth linearizzate:

```

float sceneZ = SampleLinearSceneDepth(uv);
float actorZ = SampleLinearActorDepth(uv);

bool hasActor =
    SampleActorID(uv) == _HighlightedActorID;

// Actor più lontano del muro visibile dalla camera.
float occluded =
    hasActor && (actorZ > sceneZ + _DepthEpsilon)
    ? 1.0
    : 0.0;

// IMPORTANTISSIMO:
// questo arriva dal sistema tattico,
// non dal renderer.
float legallyVisible =
    SampleGameplayVisibility(actorWorldPos);

float silhouette =
    occluded * legallyVisible;

```

Per creare l'outline, dilata la maschera ID:

```

float center = SampleActorMask(uv);

float neighbors = 0;

neighbors = max(neighbors, SampleActorMask(uv + float2( px, 0)));
neighbors = max(neighbors, SampleActorMask(uv + float2(-px, 0)));
neighbors = max(neighbors, SampleActorMask(uv + float2(0,  py)));
neighbors = max(neighbors, SampleActorMask(uv + float2(0, -py)));

float outline =
    saturate(neighbors - center);

finalColor =
    lerp(finalColor,
        _HighlightColor.rgb,
        outline * silhouette);

```

Con pipeline reversed-Z, MSAA o depth non linearizzata il confronto va adattato; il principio rimane separare `cameraOccluded` da `gameplayVisible`.

Equivalente Unity Shader Graph

Una versione world-space richiede sostanzialmente:

```

Position [World]
    ↓
Split
    ↓
XZ
    ↓
Subtract MapMin
    ↓
Divide MapSize
    ↓
Sample Texture 2D [FoW]
    ↓
Smoothstep
    ↓
Lerp
Fog Color ↔ Scene/Base Color

```

Per un post-process:

```

Screen Position
    +
Scene Depth
    ↓
Reconstruct World Position

```

```

      ↓
Map world XZ → FoW UV
      ↓
Sample FoW
      ↓
Smoothstep / Noise / Lerp

```

Il nodo Scene Depth ufficiale di Shader Graph legge la depth texture della camera; su URP bisogna assicurarsi che la depth texture sia disponibile. ¹⁶

In Unreal Material Editor la struttura concettuale è analoga:

```

Absolute World Position
      ↓
mask XY
      ↓
subtract MapOrigin
      ↓
divide MapExtent
      ↓
Texture Sample FoW
      ↓
SmoothStep / Power / Noise
      ↓
Lerp fogged ↔ normal

```

Casi studio XCOM-like e codice open source

XCOM: Enemy Unknown e XCOM 2

L'elemento più interessante di *XCOM* non è soltanto che esista una fog of war, ma che **la visibilità sia parte della pianificazione dell'azione**.

Per *XCOM: Enemy Unknown*, le descrizioni pubbliche del gameplay indicano che la fog nasconde alieni e relative azioni finché i soldati non sono entro portata e con linea di vista. ²¹

In *XCOM 2* la telemetria della LoS diventa ancora più esplicita. Il manuale documenta che:

- il numero di nemici nella LoS del soldato è mostrato nell'HUD;
- le celle sotto LoS nemica sono contrassegnate con un'icona a forma di occhio;
- entrarvi può rompere il concealment;
- la LoS entra anche nelle regole di Overwatch;
- la maggior parte delle mappe tattiche è procedurally generated. ¹

È un'indicazione progettuale mega importante: in un sistema simile non basta calcolare cosa si vede **dopo** il movimento. Devi riuscire a interrogare la visibilità da posizioni candidate prima che il giocatore confermi la mossa.

Per questo una buona API interna sarebbe:

```
VisibilityResult QueryVisibility(
    UnitId observer,
    GridPosition hypotheticalPosition,
    VisibilityQueryFlags flags);
```

e non semplicemente:

```
bool IsCurrentlyOnScreen(Actor actor);
```

Sul piano del level design, Firaxis ha inoltre presentato a GDC 2018 un talk specifico sulla transizione di *XCOM 2* verso un sistema di level design procedurale. ²²

Inferenza progettuale: con mappe assemblate proceduralmente, una visibility mask runtime basata sulla geometria effettivamente generata è generalmente più robusta di una soluzione basata esclusivamente su informazioni di visibilità curate a mano per ogni livello. Questa è un'inferenza ingegneristica dal sistema procedurale documentato, non una disclosure di Firaxis sul renderer. ²³

Ed è importante essere rigorosi qui: **non ho trovato nelle fonti Firaxis/GDC pubbliche consultate una descrizione low-level dello shader o dell'algoritmo FoW effettivamente usato internamente da XCOM 1/2**. Quindi non attribuirei a Firaxis, senza prove, un particolare blur, render target o algoritmo FOV.

Esiste però un'ottima implementazione indipendente che dichiara di voler riprodurre quel tipo di resa in Unreal: minmax usa una texture grid-based, un pixel per cella visibile, materiale di fog, filtering e un secondo render target per il fade temporale. È utile come **reference implementation di un effetto XCOM-like**, non come prova dell'implementazione originale di XCOM. ⁵

Phoenix Point

Phoenix Point offre un caso ancora più interessante perché la nebbia non è soltanto UI.

Nel blog ufficiale della Backer Build Three, Snapshot Games descrive la prima implementazione di **The Mist**: la nebbia si diffonde lentamente sulla mappa, blocca la linea di vista del giocatore e permette al nemico di vedere le unità che vi entrano in contatto. ²⁴

Questa è una differenza architetturale notevole:

XCOM-like fog classica:
visibility state → visual fog

Phoenix Point Mist:
world phenomenon
↓
gameplay visibility rules
+
visual phenomenon

In un sistema del genere conviene avere una **density/gameplay field** indipendente dal materiale volumetrico:

```
MistGameplayGrid
↓
LoS modifier / detection
↓
MistVisualField
↓
particles / meshes / volume
```

Così il frame rate o il LOD della nebbia non possono alterare chi vede chi.

La documentazione pubblica di Snapshot descrive il comportamento tattico, non gli shader interni; qualsiasi affermazione su una specifica implementazione volumetrica di *Phoenix Point* sarebbe quindi speculativa. ²⁴

Mutant Year Zero: Road to Eden

Mutant Year Zero è utile perché dimostra che un XCOM-like non deve per forza comunicare la visibilità principalmente con una grande coperta nera.

In un'intervista ufficiale ospitata da Epic, Haraldur Thormundsson spiega che lo stealth è centrale: il giocatore gira intorno agli accampamenti, studia i movimenti degli avversari e cerca eliminazioni isolate prima dello scontro. Lo stesso team descrive *Mutant Year Zero* come una combinazione di esplorazione real-time e tattica turn-based e conferma l'uso di Unreal Engine 4 per costruire gli ambienti. ²⁵

Il lesson learned è che, in un gioco stealth-tactical, puoi sostituire parte della fog geografica con una **visualizzazione esplicita dello spazio di rilevamento**:

conoscenza del livello	→ ambiente leggibile
conoscenza del nemico	→ detection area / cone
rischio della posizione	→ preview
stato di allerta	→ UI/VFX

Questo può essere molto più leggibile di annerire l'intera mappa quando l'obiettivo principale è pianificare un'imboscata.

Invisible, Inc.

Invisible, Inc. è forse il caso più estremo di "readability first".

Il talk GDC 2015 di James Lantz/Klei è dedicato specificamente alla progettazione dello **stealth procedurale** del gioco e alle iterazioni necessarie per far funzionare le sue meccaniche. ²⁶

La lezione utile per il rendering non è "Klei usa shader X", perché il talk pubblico non documenta una pipeline shader specifica. È piuttosto che in un gioco stealth deterministico la visualizzazione dovrebbe rispondere in modo chiaro alla domanda:

“Se mi metto qui, chi mi può vedere e perché?”

Da qui il pattern di design:

```
LoS esatta
  ↓
visualizzazione geometrica esplicita
  ↓
cone / danger area / observed tiles
  ↓
transizione grafica solo decorativa
```

In altre parole, per un *Invisible, Inc.-like* sacrificerei volentieri un po' di realismo atmosferico in cambio di bordi della visibilità estremamente interpretabili.

libtcod

Per studiare concretamente gli algoritmi di FOV, **libtcod** è una delle reference open-source più utili. Il progetto offre field-of-view e pathfinding e documenta diverse famiglie: basic raycast, diamond, shadow, permissive, restrictive e symmetric shadowcast. ²⁷

È soprattutto utile per confrontare cosa succede negli edge case:

```
# muro
@ osservatore
. pavimento

#####
#@...#
###...#
#...#
```

Diversi algoritmi possono non concordare su quali celle dietro gli spigoli siano visibili. In un tattico competitivo o molto deterministico questa scelta è parte delle regole del gioco, non un semplice dettaglio tecnico. La variante symmetric shadowcasting è stata esplicitamente progettata per proprietà di simmetria e consistenza. ¹¹

MangoFog per Unity

MangoFog è un esempio open-source di fog of war per Unity con rendering 2D/3D, MeshRenderer, SpriteRenderer o rendering diretto GPU. Il README dichiara supporto sia a reveal radiale sia LoS/FOV e indica compatibilità testata con URP e Built-In; specifica inoltre che il progetto non è pensato come soluzione mobile ottimizzata. ²⁸

Particolarmente interessante è il suo esperimento con **chunk multipli** per grandi aree: l'autore documenta un problema di seam visibili quando revealers attraversano il confine dei chunk usando il blur. È un caso reale di un trade-off molto comune nelle FoW: la suddivisione riduce la dimensione delle texture/update, ma il filtraggio deve essere coerente ai bordi. ²⁸

VaFogOfWar per Unreal

VaFogOfWar è una reference UE4 open source per top-down games. Il progetto, oggi archiviato, espone tre layer separati — current visibility, global visibility e terrain — supporta più livelli di altezza e ostacoli dinamici/statici. ²⁰

La divisione in layer è esattamente quella che consiglierei anche oggi:

```
R = current
G = explored/global
B = optional terrain/height/blocker
A = free/metadata
```

Le affermazioni del README sulle performance, incluse quelle mobile, vanno considerate self-reported dall'autore e non benchmark indipendenti. ²⁹

Confronto delle tecniche e trade-off

Calcolo della visibilità

Le valutazioni seguenti sono una sintesi ingegneristica; “costo” è relativo alle altre tecniche e dipende ovviamente da dimensioni della mappa, numero di osservatori e frequenza degli aggiornamenti. Gli algoritmi FOV disponibili in libtcod, la letteratura sui visibility polygon e il capitolo *Game AI Pro* sulla visibilità tattica forniscono le principali basi tecniche del confronto. ³⁰

Tecnica	Qualità/ precisione	Costo runtime	Dinamica	Multi-floor/ 3D	Complessità implementativa	Unity	Unreal
Raycasting	Alta sul campione testato	Medio- alto con molti ray	Eccellente	Eccellente	Bassa	Ottima	Ottima
Shadowcasting	Alta su griglia	Basso	Eccellente	Limitata	Media	Ottima	Ottima
Flood fill	Bassa come LoS, alta come connettività	Molto basso	Eccellente	Buona se graph- based	Bassa	Ottima	Ottima
Visibility polygon	Molto alta in 2D	Medio	Buona	Debole	Medio-alta	Buona	Buona
GPU occlusion query	Alta per camera rendering	GPU/ asincrono	Eccellente	Eccellente	Media	Engine- dependent	Nativo

Tecnica	Qualità/ precisione	Costo runtime	Dinamica	Multi-floor/ 3D	Complessità implementativa	Unity	Unreal
Precomputed visibility/PVS	Alta per configurazione baked	Molto basso runtime	Debole	Buona se precalcolata	Alta pipeline	Custom	Nativo per culling
Navmesh exposure	Media-alta, dipende dal sampling	Basso- medio	Buona	Buona	Alta	Ottima	Ottima

Le hardware occlusion query di Unreal sono esplicitamente un meccanismo di rendering/culling; la sua precomputed visibility scambia runtime rendering cost con memoria e preprocessing. ⁴

Per il navmesh, *Game AI Pro* evidenzia che si possono assegnare costi maggiori ai nodi esposti al nemico e discute sia raycast runtime sia visibility lookup precalcolati; la densità e disposizione dei sample influenzano direttamente precisione, memoria e costo. ³¹

Rendering della fog

Tecnica grafica	Qualità	GPU cost	Problema tipico	Unity	Unreal	Custom
Hard binary mask	Bassa/ media	Minimo	bordo artificiale	★★★★★	★★★★★	★★★★★
Bilinear/ blur mask	Buona	Basso	leakage oltre muri	★★★★★	★★★★★	★★★★★
SDF edge	Molto buona	Basso- medio	generazione/ update SDF	★★★★★	★★★★★	★★★★☆
World- space material/ decals	Molto buona	Basso- medio	mapping piani/ verticalità	★★★★★	★★★★★	★★★★★
Depth post- process	Molto buona	Medio	trasparenze/ depth edge	★★★★★	★★★★★	★★★★☆
Dissolve + noise	Alta/ stylized	Basso	shimmer temporale	★★★★★	★★★★★	★★★★★
Silhouette via depth/ ID	Alta/ leggibile	Medio	hidden-info leak	★★★★☆	★★★★★	★★★★☆
Particles	Alta localmente	Medio	overdraw	★★★★★	★★★★★	★★★★☆
True volumetric fog	Molto alta	Alto	froxel/ volume cost	pipeline- dependent	★★★★★	★★★★☆☆

Il problema di leakage del semplice blur è documentato concretamente nell'implementazione XCOM-like di minmax. ¹⁵ La depth texture è direttamente accessibile in Shader Graph e può alimentare effetti screen-space, mentre i Soft Particles di URP mostrano un uso ufficiale della depth per sfumare intersezioni. ³² Per il volumetrico, Unreal documenta esplicitamente il rapporto fra risoluzione del volume, numero di slice Z, qualità e costo GPU. ¹⁸

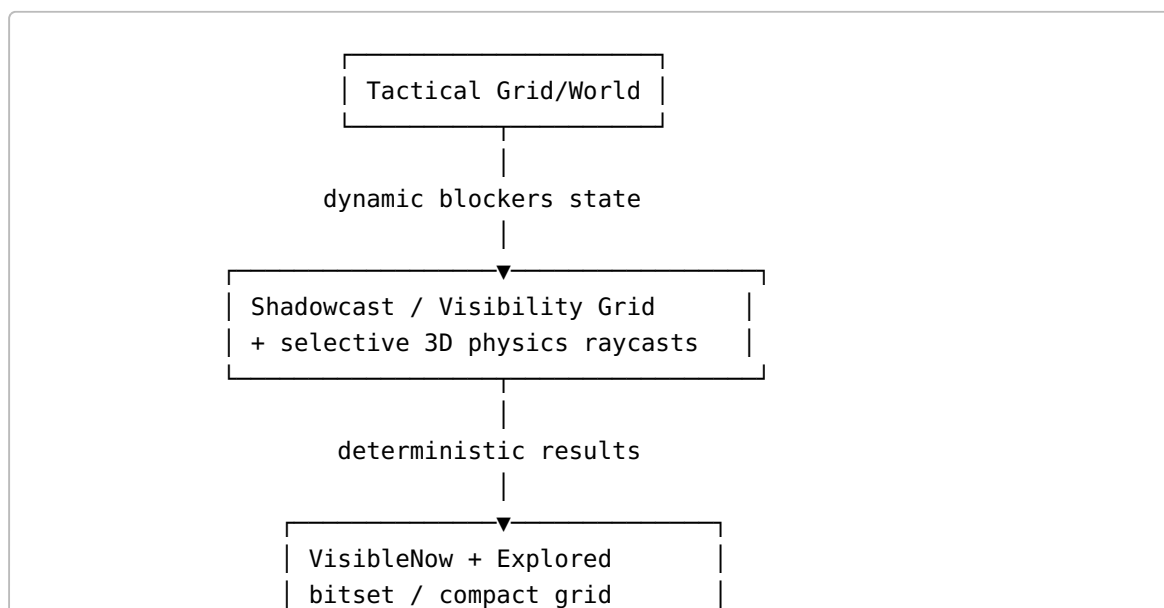
Screen-space contro world-space

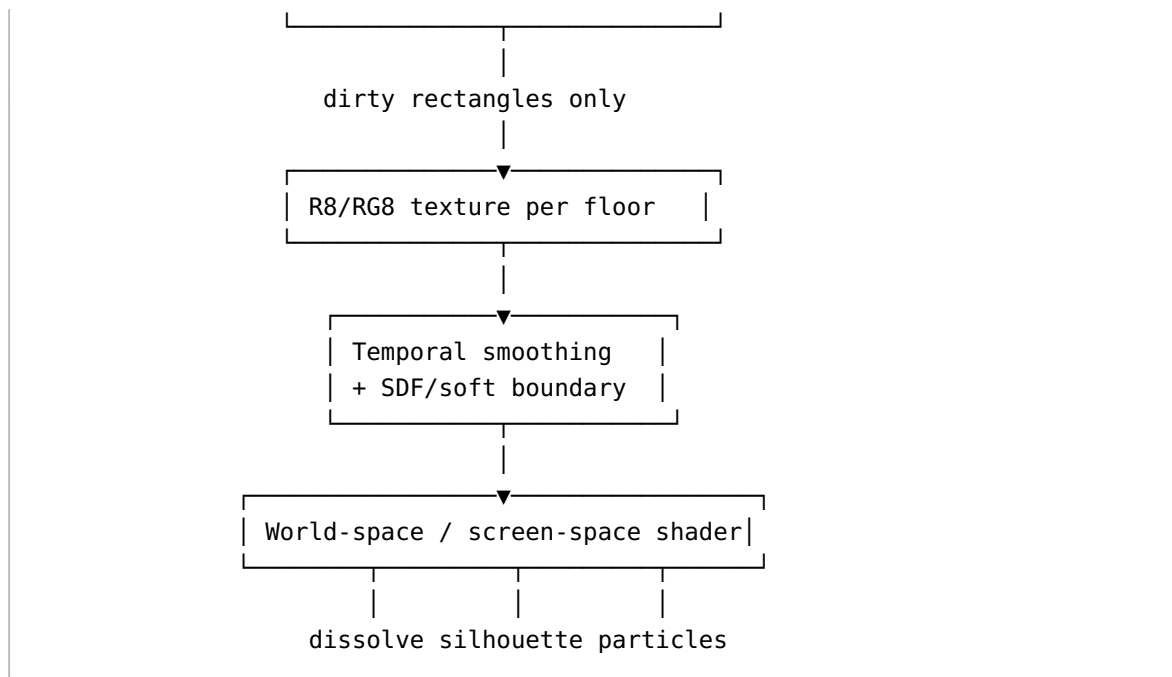
Aspetto	World-space FoW	Screen-space FoW
Stabilità muovendo camera	Eccellente	Buona se ricostruzione corretta
Minimap	Naturale	Richiede secondo sistema
Trasparenze	Material-dependent	Problematiche se assenti dalla depth
Multi-camera	Naturale	Pass per camera
Integrazione	Material/decal	Un solo fullscreen pass
Effetti depth	Richiede depth extra	Naturali
Oggetti verticali	Richiede decisione di mapping	Vengono trattati pixel-per-pixel
Debug tattico	Molto semplice	Meno intuitivo

Per un tattico isometrico personalmente sceglierei **world-space mask come dato canonico GPU** e, se serve, un post-process screen-space come secondo stadio. In questo modo il post-process non possiede lo stato della FoW: lo visualizza soltanto.

Raccomandazioni operative e fonti chiave

Per un gioco tattico a turni senza vincoli hardware specifici, partirei da questa architettura:





Per un XCOM-like classico, userei shadowcasting simmetrico per il FOV di area e raycast 3D selettivi per bersagli, cover, finestre e situazioni verticali. Questo combina l'efficienza di un FOV grid-based con una verifica geometrica precisa dove davvero conta. Le famiglie di algoritmi di libtcod e le proprietà del symmetric shadowcasting sono ottimi riferimenti implementativi. ³³

Per livelli con muri ad angoli arbitrari, userei un visibility polygon 2D rasterizzato nella visibility texture, eventualmente seguito da raycast 3D solo per elementi che non possono essere rappresentati nella planimetria. La letteratura sui visibility polygon fornisce una base formale e implementazioni efficienti del problema. ³

Per livelli pienamente 3D e multi-floor, darei priorità ai raycast/shape query e a celle di visibilità stratificate per piano. Il navmesh lo userei per scegliere *dove vale la pena testare* e per costruire mappe di esposizione AI, non come unica sorgente della LoS. Il pattern di visibility-weighted tactical pathfinding è documentato in *Game AI Pro*. ³¹

Per mappe indoor quasi statiche, una PVS precomputata può essere un ottimo broad phase: prima recuperi l'insieme di aree potenzialmente visibili, poi esegui LoS precisa soltanto su quelle. Il modello è simile alle visibility cell offline documentate da Unreal. ⁸

Per la grafica, terrei una texture world-space a bassa risoluzione come dato primario. Un texel per tile è una baseline sensata e viene utilizzata anche nella reference implementation UE4 XCOM-like di minmax; se il bordo appare troppo squadrato, aumenterei prima la risoluzione della maschera o introdurrei una distanza dal bordo, piuttosto che applicare un blur enorme. ⁵

Gli update dovrebbero essere event-driven. In un turn-based non c'è quasi mai motivo di ricalcolare l'intera LoS a 60/120 Hz. Le occasioni naturali per invalidare la cache sono movimento di un'unità, apertura/chiusura di una porta, distruzione di cover, comparsa di smoke/mist, cambio piano o modifica del range sensore. Questo sfrutta direttamente il fatto che lo stato tattico cambia in eventi discreti anziché continuamente.

Userei dirty rectangles/chunk solo quando necessario. MangoFog mostra che il chunking può consentire di gestire aree grandi, ma documenta anche seam visibili con il blur lungo i confini: padding/ halo texels fra chunk sono quindi essenziali se il filtro campiona i vicini. ²⁸

Il temporal smoothing deve essere grafico. Un nemico dovrebbe diventare `VisibleNow=true` immediatamente nel gameplay, anche se il dissolve impiega 150 ms a completarsi. La doppia Render Target dimostrata da minmax è un buon pattern per mantenere separato stato istantaneo e stato mostrato. ¹⁵

Le silhouette vanno sempre filtrate con la LoS tattica. Il depth test può dirti che un personaggio è dietro un muro rispetto alla camera, ma soltanto il sistema gameplay può dirti se il giocatore è autorizzato a sapere che si trova lì.

Il volumetrico dovrebbe essere seasoning, non il piatto principale. Unreal permette densità volumetrica globale e locale e documenta un costo dipendente dalla risoluzione della griglia volumetrica; in un tattico conviene quindi riservare il volumetrico alle fasce di frontiera, a eventi come la Mist di *Phoenix Point* o a momenti cinematografici, lasciando alla mask 2D/2.5D la responsabilità informativa. ³⁴

Per il debugging costruirei almeno tre visualizzazioni indipendenti:

```
F1: gameplay cells
    verde = visible
    grigio = explored
    nero = unknown

F2: blockers
    rosso = opaque
    giallo = partial
    blu = dynamic

F3: GPU FoW
    visualizzazione diretta della R8/RG8
```

Se `F1` è corretto ma il gioco mostra un artefatto, hai un problema shader. Se `F1` è già sbagliato, non ha senso toccare blur o post-processing. Questa separazione fa risparmiare una quantità assurda di tempo durante lo sviluppo.

Stack consigliati per scenario

Tipo di gioco	Visibility core	Rendering	Extra
XCOM-like standard	symmetric shadowcast + selective raycast	RG8 world-space + SDF/soft fade	LoS preview, silhouette
Invisible Inc.-like	grid FOV molto deterministico	hard/soft geometric overlay	vision cones espliciti
Mutant Year Zero-like	raycast + detection graph	poca FoW geografica	detection zones/ stealth UI

Tipo di gioco	Visibility core	Rendering	Extra
Phoenix Point-like	raycast/grid + environmental visibility fields	mask + world fog	Mist/smoke gameplay-driven
Top-down con muri non-grid	visibility polygon	raster polygon → RT	SDF edge
Grande mappa statica	PVS/broad phase + local LoS	chunked mask	dirty updates
Multi-floor 3D	layered cells + raycasts	Texture2DArray	floor-aware depth masks

Il punto di partenza più equilibrato, in assenza di altri vincoli, è quindi:

griglia logica deterministica + symmetric shadowcasting per l'area + raycast per le verifiche 3D + RG8 current/explored per piano + world-space sampling + SDF/smoothstep + temporal ping-pong + silhouette depth/ID gated dalla gameplay LoS.

È abbastanza economico per un tattico a turni, supporta bene mappe procedurali e distruttibili, evita di accoppiare le regole alla GPU e lascia moltissimo spazio all'art direction: puoi passare da una FoW quasi board-game a una nebbia sci-fi con dissolve e volumetric wisps senza toccare una riga delle regole tattiche. Gli approcci open-source Unity/Unreal e le API FOV di libtcod dimostrano la praticità di questa separazione in implementazioni reali. ³⁵

Fonti primarie e ufficiali più utili

Fonte	Tipo	Perché è particolarmente utile
Bungiu et al., "Efficient Computation of Visibility Polygons" ³	Paper accademico	definizione formale e algoritmi per visibility polygon
Daniel Brewer, "Tactical Pathfinding on a NavMesh", Game AI Pro ¹³	Capitolo tecnico originale	exposure, visibility lookup, raycast e navmesh tattico
libtcod FOV ³⁶	Repo/documentazione open source	raycast, shadowcasting, permissive/restrictive FOV
XCOM 2 Manual ¹	Documentazione ufficiale/licenziata	comportamento della LoS, concealment, mappe procedurali
Firaxis/GDC, XCOM 2 procedural level design ²²	Talk GDC	contesto di level generation dinamica
Phoenix Point, Backer Build Three ²⁴	Blog ufficiale developer	Mist come fenomeno che influenza realmente LoS
Klei/GDC, "Designing Procedural Stealth for Invisible, Inc." ²⁶	Talk GDC	progettazione di stealth deterministico/procedurale

Fonte	Tipo	Perché è particolarmente utile
The Bearded Ladies / Epic, Mutant Year Zero ²⁵	Developer interview ufficiale	stealth + tactical gameplay in UE4
Unity Shader Graph Scene Depth ¹⁶	Documentazione ufficiale	screen-space/depth-based FoW
Unity URP Soft Particles ¹⁷	Documentazione ufficiale	depth fade e intersezione con geometria
Unreal Visibility & Occlusion Culling ⁷	Documentazione ufficiale	distinzione tra occlusion rendering e LoS gameplay
Unreal Precomputed Visibility ⁸	Documentazione ufficiale	PVS/cell visibility e relativi trade-off
Unreal Volumetric Fog ¹⁹	Documentazione ufficiale	froxel/volume fog, qualità e costo
MangoFog Unity ²⁸	Repo open source	LoS, blur, GPU rendering, chunking
VaFogOfWar UE4 ²⁰	Repo open source	current/global visibility, height e dynamic blockers
minmax, grid-based UE4 FoW ⁵	Devlog tecnico con materiale/Blueprint	reference implementation molto vicina al look XCOM-like

¹ ²³ <https://www.feralinteractive.com/en/manual/xcom2/latest/steam/>
<https://www.feralinteractive.com/en/manual/xcom2/latest/steam/>

² ⁶ ⁹ ³⁰ https://libtcod.github.io/docs/html2/fov_compute.html
https://libtcod.github.io/docs/html2/fov_compute.html

³ <https://arxiv.org/pdf/1403.3905>
<https://arxiv.org/pdf/1403.3905>

⁴ ⁷ <https://dev.epicgames.com/documentation/unreal-engine/visibility-and-occlusion-culling-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/visibility-and-occlusion-culling-in-unreal-engine>

⁵ ¹⁴ ¹⁵ <https://minmax.itch.io/operators/devlog/222177/rendering-grid-based-fog-of-war-in-unreal-engine-4-part-1>
<https://minmax.itch.io/operators/devlog/222177/rendering-grid-based-fog-of-war-in-unreal-engine-4-part-1>

⁸ <https://dev.epicgames.com/documentation/unreal-engine/precomputed-visibility-volumes-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/precomputed-visibility-volumes-in-unreal-engine>

¹⁰ ¹¹ ³³ <https://www.albertford.com/shadowcasting/>
<https://www.albertford.com/shadowcasting/>

¹² <https://github.com/trylock/visibility>
<https://github.com/trylock/visibility>

- 13 31 https://www.gameaipro.com/GameAIPro/GameAIPro_Chapter27_Tactical_Pathfinding_on_a_NavMesh.pdf
https://www.gameaipro.com/GameAIPro/GameAIPro_Chapter27_Tactical_Pathfinding_on_a_NavMesh.pdf
- 16 32 <https://docs.unity3d.com/Packages/com.unity.shadergraph%4017.0/manual/Scene-Depth-Node.html>
<https://docs.unity3d.com/Packages/com.unity.shadergraph%4017.0/manual/Scene-Depth-Node.html>
- 17 <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal%4013.1/manual/particles-lit-shader.html>
<https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal%4013.1/manual/particles-lit-shader.html>
- 18 19 34 <https://dev.epicgames.com/documentation/unreal-engine/volumetric-fog-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/volumetric-fog-in-unreal-engine>
- 20 29 <https://github.com/ufna/VaFogOfWar>
<https://github.com/ufna/VaFogOfWar>
- 21 https://en.wikipedia.org/wiki/XCOM%3A_Energy_Unknown
https://en.wikipedia.org/wiki/XCOM%3A_Energy_Unknown
- 22 <https://www.gamedeveloper.com/>
<https://www.gamedeveloper.com/>
- 24 <https://phoenixpoint.info/blog/2018/11/7/backer-build-three>
<https://phoenixpoint.info/blog/2018/11/7/backer-build-three>
- 25 <https://www.unrealengine.com/developer-interviews/mutant-year-zero---from-pen-and-paper-rpg-to-pixel-perfect-post-apocalypse>
<https://www.unrealengine.com/developer-interviews/mutant-year-zero---from-pen-and-paper-rpg-to-pixel-perfect-post-apocalypse>
- 26 <https://www.gamedeveloper.com/design/video-designing-procedural-stealth-for-i-invisible-inc-i>
<https://www.gamedeveloper.com/design/video-designing-procedural-stealth-for-i-invisible-inc-i>
- 27 36 <https://github.com/libtcod/libtcod-fov>
<https://github.com/libtcod/libtcod-fov>
- 28 35 <https://github.com/mkwozniak/mangofogunity>
<https://github.com/mkwozniak/mangofogunity>